CPL 105 – Root Finding

In [1]:
```
# Q1 II.)
```

In [2]:
```
def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
       print("The function must have opposite signs at a and b.")

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c

        if f(a) * f(c) < 0:
            b = c
        else:
            a = c

    return (a + b) / 2
```

In [ ]:
```
# Q1 III) a.
```

In [3]:
```
import numpy as np

def f(x):
    return x**2 - np.exp(-x) - 2 * np.sin(x)

def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("The function must have opposite signs at a and b.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2


root = bisection(f, 0, 2, 1e-6)
print("Root of f(x): ", root)
```

```
Root of f(x):  1.489588737487793
```

In [4]:
```
# Q1 III) b.
```

In [5]:
```
import numpy as np

def f(t):
    return 10 * t - 4.9 * t**2

# Bisection method
def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("Bisection method fails. f(a) and f(b) must have opposite signs.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        elif f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2

root = bisection(f, 0.1, 3, 1e-6)
print("Time when projectile returns to initial height:", root, "seconds")
```

```
Time when projectile returns to initial height: 2.040816044807434 seconds
```

In [6]:
```
# Q1 III) c.
```

In [7]:
```
import numpy as np

def f(x):
    return x**2 - 3

def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("f(a) and f(b) must have opposite signs.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        elif f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2

root = bisection(f, 1, 2, 1e-6)
print("Approximate sqrt(3):", root)
```

```
Approximate sqrt(3): 1.732050895690918
```

In [ ]:
```
# Q1 III) d.
```

In [8]:
```
import numpy as np

def f(L):
    return 2 * np.pi * np.sqrt(L / 9.8) - 10


def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("f(a) and f(b) must have opposite signs.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        elif f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2


length = bisection(f, 1, 100, 1e-6)
print("Length L for T = 10 seconds:", length, "meters")
```

```
Length L for T = 10 seconds: 24.82368966192007 meters
```

In [ ]:
```
# Q1 III) e.
```

In [9]:
```
import numpy as np

def f(t):
    return 5 * (1 - np.exp(-t / 10)) - 2.5

def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("f(a) and f(b) must have opposite signs.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        elif f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2

time = bisection(f, 0, 50, 1e-6)
print("Time at which Vc(t) = 2.5 V:", time, "seconds")
```

```
Time at which Vc(t) = 2.5 V: 6.931471079587936 seconds
```

In [ ]: